```
from __future__ import annotations
import math, re, warnings
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo
import numpy as np
import pandas as pd
import streamlit as st
import yfinance as yf
import plotly.express as px
import plotly.graph_objects as go

try:
    import feedparser
except Exception:
    feedparser = None
try:
    import requests
except Exception:
    requests = None

# --- Rate-limit / cache session for yfinance (graceful fallback) ---
try:
    from requests import Session
    from requests_cache import CacheMixin, SQLiteCache
    from requests_ratelimiter import LimiterMixin, MemoryQueueBucket
    from pyrate_limiter import Duration, RequestRate, Limiter

    class CachedLimiterSession(CacheMixin, LimiterMixin, Session):
        pass

    @st.cache_resource
    def get_yf_session():
        limiter = Limiter(
            RequestRate(60, Duration.MINUTE),
            RequestRate(360, Duration.HOUR),
        )
        return CachedLimiterSession(
            limiter=limiter,
            bucket_class=MemoryQueueBucket,
            backend=SQLiteCache("yfinance.cache"),
        )

    _YF_SESSION = get_yf_session()
except Exception:
    _YF_SESSION = None

st.set_page_config(page_title="ATLAS Analytics", page_icon="🌐", layout="wide")
TZ = ZoneInfo("Europe/Berlin")
```

now = datetime.now(TZ)

STOCKS = {
"AAPL":"Apple","MSFT":"Microsoft","NVDA":"NVIDIA","AMZN":"Amazon","GOOGL":"Alphabet A","GOOG":"Alphabet C","META":"Meta Platforms","BRK-B":"Berkshire Hathaway B","LLY":"Eli Lilly","AVGO":"Broadcom","JPM":"JPMorgan Chase","V":"Visa","MA":"Mastercard","XOM":"Exxon Mobil","COST":"Costco","WMT":"Walmart","NFLX":"Netflix","ORCL":"Oracle","AMD":"AMD","CRM":"Salesforce","ADBE":"Adobe","QCOM":"Qualcomm","INTC":"Intel","TSLA":"Tesla","KO":"Coca-Cola","PEP":"PepsiCo","UNH":"UnitedHealth","CAT":"Caterpillar","GE":"GE Aerospace","IBM":"IBM","CSCO":"Cisco","AMAT":"Applied Materials","MU":"Micron","PLTR":"Palantir","UBER":"Uber","ABNB":"Airbnb","DIS":"Walt Disney","MCD":"McDonald's","BA":"Boeing","GS":"Goldman Sachs","BAC":"Bank of America","C":"Citigroup","WFC":"Wells Fargo","CVX":"Chevron","COP":"ConocoPhillips","PFE":"Pfizer","MRK":"Merck","TMO":"Thermo Fisher","ISRG":"Intuitive Surgical","NOW":"ServiceNow","PANW":"Palo Alto Networks","SNPS":"Synopsys","CRWD":"CrowdStrike","SHOP":"Shopify","DELL":"Dell Technologies","ARM":"Arm Holdings","SMCI":"Super Micro Computer","LRCX":"Lam Research","KLAC":"KLA","TXN":"Texas Instruments","BKNG":"Booking Holdings","SPOT":"Spotify","PYPL":"PayPal","T":"AT&T","VZ":"Verizon","NKE":"Nike","LOW":"Lowe's","HD":"Home Depot","UPS":"UPS","FDX":"FedEx","AXP":"American Express","BLK":"BlackRock","SCHW":"Charles Schwab","COIN":"Coinbase","MSTR":"Strategy","F":"Ford","GM":"General Motors",
"SAP":"SAP","SIE.DE":"Siemens","ALV.DE":"Allianz","DTE.DE":"Deutsche Telekom","MUV2.DE":"Munich Re","AIR.PA":"Airbus","MC.PA":"LVMH","OR.PA":"L'Oreal","TTE.PA":"TotalEnergies","SAN.PA":"Sanofi","SU.PA":"Schneider Electric","ASML":"ASML","RHM.DE":"Rheinmetall","MBG.DE":"Mercedes-Benz","BMW.DE":"BMW","VOW3.DE":"Volkswagen","BAS.DE":"BASF","IFX.DE":"Infineon","ADS.DE":"Adidas",
"DBK.DE":"Deutsche Bank","ENR.DE":"Siemens Energy","BN.PA":"Danone","NOVO-B.CO":"Novo Nordisk","NVO":"Novo Nordisk ADR","AZN":"AstraZeneca","SHEL":"Shell","HSBA.L":"HSBC","BP.L":"BP","ULVR.L":"Unilever","RIO":"Rio Tinto","GSK":"GSK","REL.L":"RELX","LSEG.L":"London Stock Exchange Group","BHP":"BHP Group","NESN.SW":"Nestle","ROG.SW":"Roche","NOVN.SW":"Novartis","UBSG.SW":"UBS",
"7203.T":"Toyota","6758.T":"Sony","9984.T":"SoftBank Group","005930.KS":"Samsung Electronics","000660.KS":"SK Hynix","0700.HK":"Tencent","9988.HK":"Alibaba","3690.HK":"Meituan","TSM":"TSMC ADR","BABA":"Alibaba ADR","INFY":"Infosys ADR","RELIANCE.NS":"Reliance Industries","^BSESN":"BSE Sensex","^N225":"Nikkei 225",
"ACN":"Accenture","ADP":"Automatic Data Processing","ADI":"Analog Devices","AIG":"American International Group","AMGN":"Amgen","APH":"Amphenol","APP":"AppLovin","APO":"Apollo Global Management","AXON":"Axon Enterprise","BKR":"Baker Hughes","BMY":"Bristol Myers Squibb","BX":"Blackstone","CARR":"Carrier Global","CCL":"Carnival","CEG":"Constellation Energy","CHTR":"Charter Communications","CMCSA":"Comcast","CMG":"Chipotle Mexican Grill","CMI":"Cummins","COR":"Cencora","CPRT":"Copart","CRH":"CRH","CTAS":"Cintas","CVS":"CVS Health","DASH":"DoorDash","DDOG":"Datadog","DE":"Deere","DG":"Dollar

General","DHR":"Danaher","DLR":"Digital Realty","DUK":"Duke Energy","EA":"Electronic Arts","EBAY":"eBay","ECL":"Ecolab","ELV":"Elevance Health","EMR":"Emerson Electric","ETN":"Eaton","FIS":"Fidelity National Information Services","FTNT":"Fortinet","GD":"General Dynamics","GILD":"Gilead Sciences","GMED":"Globus Medical","GPN":"Global Payments","HAL":"Halliburton","HON":"Honeywell","HPE":"Hewlett Packard Enterprise","HPQ":"HP Inc.","ICE":"Intercontinental Exchange","INTU":"Intuit","ITW":"Illinois Tool Works","JCI":"Johnson Controls","KDP":"Keurig Dr Pepper","KKR":"KKR","KHC":"Kraft Heinz","KMI":"Kinder Morgan","LHX":"L3Harris Technologies","LIN":"Linde","LMT":"Lockheed Martin","LULU":"Lululemon","MAR":"Marriott International","MCHP":"Microchip Technology","MDLZ":"Mondelez International","MET":"MetLife","MMM":"3M","MNST":"Monster Beverage","MO":"Altria","MRVL":"Marvell Technology","MS":"Morgan Stanley","MSI":"Motorola Solutions","NEM":"Newmont","NOC":"Northrop Grumman","ODFL":"Old Dominion Freight Line","ON":"ON Semiconductor","OXY":"Occidental Petroleum","PCAR":"PACCAR","PDD":"PDD Holdings","PGR":"Progressive","PH":"Parker Hannifin","PLD":"Prologis","PM":"Philip Morris International","PNC":"PNC Financial Services","PSA":"Public Storage","RBLX":"Roblox","REGN":"Regeneron Pharmaceuticals","ROP":"Roper Technologies","ROST":"Ross Stores","RTX":"RTX","SBUX":"Starbucks","SLB":"SLB","SNOW":"Snowflake","SO":"Southern Company","STX":"Seagate Technology","SYK":"Stryker","TGT":"Target","TJX":"TJX Companies","TMUS":"T-Mobile US","TRV":"Travelers","TT":"Trane Technologies","TTD":"The Trade Desk","TWLO":"Twilio","U":"Unity Software","VLO":"Valero Energy","VRTX":"Vertex Pharmaceuticals","WELL":"Welltower","WM":"Waste Management","WMB":"Williams Companies","WDAY":"Workday","XEL":"Xcel Energy","ZTS":"Zoetis",
"7203.T":"Toyota Motor","6758.T":"Sony Group","8035.T":"Tokyo Electron","9983.T":"Fast Retailing","8306.T":"Mitsubishi UFJ Financial","6861.T":"Keyence","6501.T":"Hitachi","7974.T":"Nintendo","9432.T":"NTT","8058.T":"Mitsubishi Corporation","005380.KS":"Hyundai Motor","051910.KS":"LG Chem","005490.KS":"POSCO Holdings","035420.KS":"NAVER","066570.KS":"LG Electronics","000270.KS":"Kia","1810.HK":"Xiaomi","0941.HK":"China Mobile","1299.HK":"AIA Group","9618.HK":"JD.com","9999.HK":"NetEase","1024.HK":"Kuaishou","2318.HK":"Ping An Insurance","1398.HK":"ICBC","3988.HK":"Bank of China","D05.SI":"DBS Group","O39.SI":"OCBC","CBA.AX":"Commonwealth Bank of Australia","BHP.AX":"BHP Group Australia","CSL.AX":"CSL Limited","WES.AX":"Wesfarmers","NAB.AX":"National Australia Bank","RIO.AX":"Rio Tinto Australia","BARC.L":"Barclays","LLOY.L":"Lloyds Banking Group","DGE.L":"Diageo","PRU.L":"Prudential","III.L":"3i Group","AZN.L":"AstraZeneca London","BATS.L":"British American Tobacco","IBE.MC":"Iberdrola","ITX.MC":"Inditex","SAN.MC":"Banco Santander","BBVA.MC":"BBVA","ENEL.MI":"Enel","ISP.MI":"Intesa Sanpaolo","UCG.MI":"UniCredit","KER.PA":"Kering","CS.PA":"AXA","BNP.PA":"BNP Paribas","DG.PA":"Vinci","SAF.PA":"Safran","RACE.MI":"Ferrari","STLAM.MI":"Stellantis","ADYEN.AS":"Adyen","PHIA.AS":"Philips","PRX.AS":"Prosus","DSY.PA":"Dassault Systemes","BAYN.DE":"Bayer","HNR1.DE":"Hannover Rueck","HEI.DE":"Heidelberg Materials","FRE.DE":"Fresenius","BEI.DE":"Beiersdorf","ZAL.DE":"Zalando","RWE.DE":"RWE","VNA.DE":"Vonovia","MTX.DE":"MTU Aero

```
Engines","HEN3.DE":"Henkel","PUM.DE":"Puma","CON.DE":"Continental","QIA.DE":"QIAGE
N","DHL.DE":"DHL Group","DB1.DE":"Deutsche Boerse","KNIN.SW":"Kuehne +
Nagel","ZURN.SW":"Zurich
Insurance","ABBN.SW":"ABB","EQNR.OL":"Equinor","DNB.OL":"DNB
Bank","VOLV-B.ST":"Volvo
Group","ERIC-B.ST":"Ericsson","NESTE.HE":"Neste","ENI.MI":"Eni","TCS.NS":"Tata
Consultancy Services","HDFCBANK.NS":"HDFC Bank","ICICIBANK.NS":"ICICI
Bank","BHARTIARTL.NS":"Bharti Airtel","LT.NS":"Larsen & Toubro","SBIN.NS":"State Bank
of India","ITC.NS":"ITC Limited","TATAMOTORS.NS":"Tata Motors"
}
ETFS = {
"SPY":"SPDR S&P 500","VOO":"Vanguard S&P 500","QQQ":"Invesco
Nasdaq-100","VTI":"Vanguard Total US Market","VT":"Vanguard Total
World","ACWI":"iShares MSCI ACWI","VEA":"Vanguard Developed
Markets","VWO":"Vanguard Emerging Markets","IEFA":"iShares Core MSCI
EAFE","IEMG":"iShares Core MSCI Emerging Markets","EFA":"iShares MSCI
EAFE","EEM":"iShares MSCI Emerging Markets","DIA":"SPDR Dow Jones Industrial
Average","IWM":"iShares Russell 2000","SCHD":"Schwab US Dividend
Equity","VIG":"Vanguard Dividend Appreciation","XLK":"Technology Select
Sector","XLF":"Financial Select Sector","XLE":"Energy Select Sector","XLV":"Health Care
Select Sector","XLI":"Industrial Select Sector","SMH":"VanEck
Semiconductor","SOXX":"iShares Semiconductor","ARKK":"ARK Innovation","TLT":"iShares
20+ Year Treasury Bond","IEF":"iShares 7-10 Year Treasury Bond","SHY":"iShares 1-3 Year
Treasury Bond","AGG":"iShares Core US Aggregate Bond","BND":"Vanguard Total Bond
Market","LQD":"iShares Investment Grade Corporate Bond","HYG":"iShares High Yield
Corporate Bond","TIP":"iShares TIPS Bond","GLD":"SPDR Gold Shares","IAU":"iShares
Gold Trust","SLV":"iShares Silver Trust","PDBC":"Invesco Optimum Yield Diversified
Commodity","DBC":"Invesco DB Commodity Index","USO":"United States Oil
Fund","UNG":"United States Natural Gas Fund","VNQ":"Vanguard Real
Estate","REET":"iShares Global REIT","EUNL.DE":"iShares Core MSCI World
UCITS","IWDA.AS":"iShares Core MSCI World","VWCE.DE":"Vanguard FTSE All-World
UCITS","VUSA.AS":"Vanguard S&P 500 UCITS","CSPX.L":"iShares Core S&P 500
UCITS","SXR8.DE":"iShares Core S&P 500 UCITS","EXSA.DE":"iShares STOXX Europe
600","IS3N.DE":"iShares Core MSCI EM IMI","XDWT.DE":"Xtrackers MSCI World
Information Technology","IQQH.DE":"iShares Global Clean Energy UCITS","EIMI.L":"iShares
Core MSCI EM IMI"
}
COMMODITIES = {
"GC=F":"Gold futures","SI=F":"Silver futures","PL=F":"Platinum futures","PA=F":"Palladium
futures","HG=F":"Copper futures","CL=F":"WTI crude oil","BZ=F":"Brent crude
oil","NG=F":"Natural gas","ALI=F":"Aluminium futures","ZC=F":"Corn
futures","ZW=F":"Wheat futures","ZS=F":"Soybean futures","KC=F":"Coffee
futures","SB=F":"Sugar futures","CT=F":"Cotton futures","CC=F":"Cocoa
futures","LE=F":"Live cattle","HE=F":"Lean hogs","LBR=F":"Lumber futures"
}
PHYSICAL_METALS = {"GC=F":"Gold (exchange-traded futures proxy)","SI=F":"Silver
(exchange-traded futures proxy)","PL=F":"Platinum (exchange-traded futures
proxy)","PA=F":"Palladium (exchange-traded futures proxy)"}
```

```
BOND_DURATION =
{"TLT":16.0,"IEF":7.5,"SHY":1.8,"AGG":6.0,"BND":6.0,"LQD":8.0,"HYG":3.5,"TIP":6.5}


@st.cache_data(ttl=900, show_spinner=False)
def fetch_batch_history(tickers: tuple) -> dict:
    if not tickers:
        return {}
    tickers = list(tickers)
    kwargs = dict(period="max", interval="1d", auto_adjust=True, actions=False,
                  group_by="ticker", threads=True, progress=False)
    if _YF_SESSION is not None:
        kwargs["session"] = _YF_SESSION
    try:
        raw = yf.download(tickers, **kwargs)
    except Exception:
        return {}
    out = {}
    if raw is None or raw.empty:
        return out
    if isinstance(raw.columns, pd.MultiIndex):
        for t in tickers:
            try:
                df = raw[t].dropna(subset=["Close"])
                if df.empty:
                    continue
                df = df[["Close"]].copy()
                df.index = pd.to_datetime(df.index).tz_localize(None)
                out[t] = df[~df.index.duplicated(keep="last")].sort_index()
            except Exception:
                continue
    else:
        t = tickers[0]
        try:
            df = raw.dropna(subset=["Close"])[["Close"]].copy()
            df.index = pd.to_datetime(df.index).tz_localize(None)
            out[t] = df[~df.index.duplicated(keep="last")].sort_index()
        except Exception:
            pass
    return out


@st.cache_data(ttl=900, show_spinner=False)
def fetch_history(ticker: str) -> pd.DataFrame:
    return fetch_batch_history((ticker,)).get(ticker, pd.DataFrame())


def compute_metrics(ticker, name, kind, d: pd.DataFrame):
```

```
    if d is None or d.empty or len(d) < 3:
        return None
    p = d.Close.astype(float)
    r = p.pct_change().dropna()
    if r.empty:
        return None
    years = max((p.index[-1] - p.index[0]).days / 365.25, 1 / 365.25)
    total = p.iloc[-1] / p.iloc[0] - 1
    ann = (p.iloc[-1] / p.iloc[0]) ** (1 / years) - 1 if p.iloc[-1] > 0 and p.iloc[0] > 0 else np.nan
    vol = r.std(ddof=1) * np.sqrt(252)
    dd = (p / p.cummax() - 1).min()
    momentum = p.iloc[-1] / p.iloc[-min(len(p), 253)] - 1
    sharpe = (r.mean() * 252) / (r.std(ddof=1) * np.sqrt(252)) if r.std(ddof=1) > 0 else np.nan
    score = 50 + 28 * np.tanh((ann if np.isfinite(ann) else 0) * 2) \
            + 12 * np.tanh((momentum if np.isfinite(momentum) else 0) * 2) \
            - 18 * min(float(vol) if np.isfinite(vol) else 1, 1.5) \
            - 10 * min(abs(float(dd)) if np.isfinite(dd) else 1, 1.0)
    risk = "Niedrig" if vol < .15 and abs(dd) < .20 else "Mittel" if vol < .30 and abs(dd) < .40
else "Hoch"
    return {"Ticker": ticker, "Anlage": name, "Typ": kind, "Kurs": p.iloc[-1],
            "Seit Beginn %": total * 100, "Rendite p.a. %": ann * 100,
            "Volatilität %": vol * 100, "Max. Verlustphase %": dd * 100,
            "1J Rendite %": momentum * 100, "Sharpe": sharpe, "Score": score,
            "Risiko": risk, "Startdatum": p.index[0].date().isoformat(),
            "Letztes Datum": p.index[-1].date().isoformat(), "Datenpunkte": len(p)}


def get_table(universe, kind, selected=None):
    tickers = [t for t in (selected if selected is not None else universe.keys()) if t in universe]
    if not tickers:
        return pd.DataFrame()
    batch = fetch_batch_history(tuple(tickers))
    out = []
    progress = st.progress(0, text=f"Analysiere {kind} …")
    for i, t in enumerate(tickers):
        row = compute_metrics(t, universe[t], kind, batch.get(t, pd.DataFrame()))
        if row:
            out.append(row)
        progress.progress((i + 1) / max(len(tickers), 1), text=f"Prüfe {t} ({i+1}/{len(tickers)})")
    progress.empty()
    if not out:
        return pd.DataFrame()
    return pd.DataFrame(out).sort_values("Score", ascending=False).reset_index(drop=True)


HIGH_BETTER = {"Seit Beginn %", "Rendite p.a. %", "1J Rendite %", "Sharpe", "Score"}
LOW_BETTER = {"Volatilität %", "Max. Verlustphase %"}
```

```
def style_table(df):
    if df.empty:
        return df
    def colors(col):
        if col.name in HIGH_BETTER:
            best, worst = col.idxmax(), col.idxmin()
        elif col.name in LOW_BETTER:
            best, worst = col.idxmin(), col.idxmax()
        else:
            return [""] * len(col)
        return ["background-color: #c8f7c5; color: #123b12; font-weight: 600" if i == best
                else "background-color: #ffd2d2; color: #7d1010; font-weight: 600" if i == worst
                else "" for i in col.index]
    numeric = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]
    fmt = {c: "{:,.2f}" for c in numeric}
    return df.style.apply(colors, axis=0).format(fmt, na_rep="—")


def rss_news():
    if not feedparser:
        return pd.DataFrame(), "Paket feedparser fehlt."
    feeds = [
        ("BBC World", "https://feeds.bbci.co.uk/news/world/rss.xml"),
        ("BBC Technology", "https://feeds.bbci.co.uk/news/technology/rss.xml"),
        ("AP News", "https://apnews.com/hub/ap-top-news/feed"),
        ("NASA", "https://www.nasa.gov/news-release/feed/"),
        ("European Central Bank", "https://www.ecb.europa.eu/rss/press.html"),
        ("US Federal Reserve", "https://www.federalreserve.gov/feeds/press_all.xml"),
        ("European Commission",
"https://ec.europa.eu/commission/presscorner/api/rss?language=en"),
        ("SEC", "https://www.sec.gov/news/pressreleases.rss"),
        ("CNBC Top News", "https://www.cnbc.com/id/100003114/device/rss/rss.html"),
        ("WSJ Markets", "https://feeds.a.dj.com/rss/RSSMarketsMain.xml"),
    ]
    rows, errors = [], []
    cutoff = now - timedelta(days=2)
    for source, url in feeds:
        try:
            feed = feedparser.parse(url)
            for e in feed.entries[:35]:
                title = getattr(e, "title", "")
                link = getattr(e, "link", "")
                summary = re.sub("<[^>]+>", " ", getattr(e, "summary", getattr(e, "description", "")))
                raw = getattr(e, "published_parsed", None) or getattr(e, "updated_parsed", None)
                dt = datetime(*raw[:6], tzinfo=timezone.utc).astimezone(TZ) if raw else None
                if dt and dt >= cutoff:
                    rows.append({"Zeit (lokal)": dt.strftime("%d.%m.%Y %H:%M"),
```

```
                        "Quelle": source, "Meldung": title,
                        "Kurzinfo": summary.strip()[:450], "Link": link, "Zeitobjekt": dt})
        except Exception:
            errors.append(source)
    df = pd.DataFrame(rows)
    if not df.empty:
        df = df.sort_values("Zeitobjekt", ascending=False).drop_duplicates(subset=["Meldung"])
    return df, ("Nicht erreichbar: " + ", ".join(errors) if errors else "")


st.title("🌐 ATLAS Analytics")
st.caption("Markt-Screener · Langfristige Kursdaten · Risikoanalyse · Nachrichtenlage. Keine
Anlageberatung.")

with st.sidebar:
    st.header("Einstellungen")
    refresh = st.button("🔄 Daten aktualisieren")
    if refresh:
        st.cache_data.clear()
        for key in ("commodities_df", "metals_df", "idea_universe_df", "idea_df",
"idea_universe_version"):
            st.session_state.pop(key, None)
        st.rerun()
    st.caption("Kursdaten: Yahoo Finance, Aktualisierungscache 15 Minuten. Verfügbarkeit
und Historie unterscheiden sich je Ticker.")
    st.divider()
    st.subheader("Investment-Szenario")
    money = st.number_input("Anlagebetrag (€)", min_value=10.0,
max_value=10_000_000.0, value=1000.0, step=100.0, key="scenario_money")
    horizon = st.select_slider("Anlagezeitraum", options=["1 Monat","3 Monate","6 Monate","1
Jahr","3 Jahre","5 Jahre","10 Jahre"], value="5 Jahre", key="scenario_horizon")
    risk_pref = st.selectbox("Gewünschtes Risiko", ["Niedrig","Mittel","Hoch","Alle"], index=1,
key="scenario_risk")
    growth_pref = st.selectbox("Wachstumsziel", ["Stabil","Ausgewogen","Hoch","Maximal"],
index=1, key="scenario_growth")
    if "applied_scenario" not in st.session_state:
        st.session_state.applied_scenario = {"money": 1000.0, "horizon": "5 Jahre", "risk":
"Mittel", "growth": "Ausgewogen"}
    if st.button("Auf 10 Vorschläge anwenden", type="primary",
key="apply_scenario_to_suggestions"):
        st.session_state.applied_scenario = {"money": float(money), "horizon": horizon, "risk":
risk_pref, "growth": growth_pref}
        st.rerun()
    applied_scenario = st.session_state.applied_scenario
    applied_money = float(applied_scenario["money"])
    applied_horizon = applied_scenario["horizon"]
    applied_risk = applied_scenario["risk"]
    applied_growth = applied_scenario["growth"]
```

```
    risk_volatility_factor = {"Niedrig": 0.65, "Mittel": 1.00, "Hoch": 1.50, "Alle": 1.00}
    st.caption(f"Zuletzt angewendet: {applied_money:,.2f} € · {applied_horizon} · Risiko {applied_risk} · Wachstum {applied_growth}.")
    st.caption("Vorschläge sind datenbasierte Beispiele, keine persönliche Finanzberatung oder Garantie.")

all_universe = {**STOCKS, **ETFS, **COMMODITIES}
with st.spinner("Lade Marktübersichten …"):
    universe_choice = st.radio("Umfang", ["Komplette kuratierte Liste","Aktien","ETFs","Rohstoffe"], horizontal=True)
    if universe_choice == "Aktien": active = {**STOCKS}
    elif universe_choice == "ETFs": active = {**ETFS}
    elif universe_choice == "Rohstoffe": active = {**COMMODITIES}
    else: active = all_universe
    st.caption(f"Auswahl: {len(active)} Symbole aus mehreren Märkten. Eine vollständige Liste jedes weltweit gelisteten Wertpapiers ist mit dieser kostenlosen Kursquelle nicht zuverlässig verfügbar.")
    st.caption("Die Rangliste lädt automatisch. Beim ersten Start kann die große weltweite Auswahl einige Minuten benötigen; Kursdaten werden anschließend 15 Minuten zwischengespeichert.")

display_df = get_table(active, universe_choice)

st.header("Ranglisten")
st.subheader("📈 Aktien & ETFs / Gesamtmarkt")
st.write("Die Rangliste nutzt jeweils die **längste von der Datenquelle gelieferte Historie** (`period=max`). Die Score-Spalte ist eine vereinfachte Kennzahl, kein Kursziel.")
df = display_df
if not df.empty:
    types = st.multiselect("Anlagearten anzeigen", sorted(df.Typ.unique()), default=sorted(df.Typ.unique()), key="typefilter")
    view = df[df.Typ.isin(types)].copy()
    st.dataframe(style_table(view), use_container_width=True, height=540)
    pick = st.selectbox("Kursverlauf ansehen", view.Ticker.tolist(), key="chart_stock")
    h = fetch_history(pick)
    if not h.empty:
        st.plotly_chart(px.line(h.reset_index(), x=h.index.name or "Date", y="Close", title=f"{pick} – verfügbare Gesamthistorie"), use_container_width=True)
else:
    st.info("Für die ausgewählten Symbole sind derzeit keine ausreichenden Kursdaten abrufbar. Nutze bei Bedarf „Daten aktualisieren“.")

st.divider()
st.subheader("🛢️ Rohstoffe")
st.write("Rohstoffe werden überwiegend über börsengehandelte Futures abgebildet. Futures können Rollverluste, Laufzeitunterschiede und besondere Risiken aufweisen.")
if "commodities_df" not in st.session_state:
    st.session_state.commodities_df = get_table(COMMODITIES, "Rohstoffe")
```

```
cdf = st.session_state.get("commodities_df", pd.DataFrame())
if not cdf.empty:
    st.dataframe(style_table(cdf), use_container_width=True, height=520)

st.divider()
st.subheader("🥇 Physische Edelmetalle")
st.warning("Wichtig: Diese Liste verwendet Futures als Preis-Proxy für Edelmetalle, nicht den tatsächlichen Händlerpreis für Barren oder Münzen. Aufschläge, Lagerung und Kauf-/Verkaufspreise fehlen.")
if "metals_df" not in st.session_state:
    st.session_state.metals_df = get_table(PHYSICAL_METALS, "Physische Edelmetalle")
mdf = st.session_state.get("metals_df", pd.DataFrame())
if not mdf.empty:
    st.dataframe(style_table(mdf), use_container_width=True)
    mt = st.selectbox("Edelmetallkurs", mdf.Ticker.tolist(), key="metal_chart")
    mh = fetch_history(mt)
    if not mh.empty:
        fig = go.Figure()
        fig.add_trace(go.Scatter(x=mh.index, y=mh.Close, mode="lines", name=mt))
        fig.update_layout(title=f"{mt} – längste verfügbare Kursgeschichte", xaxis_title="Datum", yaxis_title="Preis (notierungsabhängig)")
        st.plotly_chart(fig, use_container_width=True)

st.divider()
st.subheader("🧱 Risiko & Portfolio")
st.subheader("Portfolio-Risiko-Rechner")
st.write("Füge Ticker und Gewichte ein. Die Gewichte werden für die Rechnung auf 100 % normiert.")
default_port = pd.DataFrame({"Ticker": ["SPY","TLT","GLD"], "Gewicht %": [50.0,30.0,20.0]})
port = st.data_editor(default_port, num_rows="dynamic", use_container_width=True, key="portfolio_editor")
conf = st.selectbox("VaR-Konfidenz", ["95 %","99 %"])
days = st.slider("VaR-Haltedauer (Tage)", 1, 252, 1)
with st.expander("⚙️ Monte-Carlo-Formel einstellen", expanded=False):
    mc_sims = st.slider("Anzahl der Simulationen", 500, 20000, 5000, step=500, key="risk_mc_sims")
    mc_days = st.slider("Simulationszeitraum (Handelstage)", 21, 1260, 252, step=21, key="risk_mc_days")
    mc_mu_factor = st.slider("Rendite-Faktor μ", 0.0, 2.0, 1.0, step=0.1, key="risk_mc_mu")
    mc_sigma_factor = st.slider("Volatilitäts-Faktor σ", 0.0, 3.0, 1.0, step=0.1, key="risk_mc_sigma")
    mc_seed = st.number_input("Zufallsstartwert (Seed)", min_value=0, max_value=999999, value=42, step=1, key="risk_mc_seed")
    st.caption("Formel: S□₊₁ = S□ · exp((μ − σ²/2)Δt + σ√Δt · Z). μ = historische mittlere Rendite, σ = historische Volatilität, Z = Zufallszahl.")
st.markdown(
    """**Kurzanleitung – Risiko & Portfolio (11 Punkte)**
```

1. **Ticker:** Kürzel der Anlage, die in die Portfolioberechnung aufgenommen wird.
2. **Gewicht %:** Anteil jeder Anlage; ATLAS normiert alle Gewichte zusammen auf 100 %.
3. **VaR-Konfidenz:** 95 % bzw. 99 % bestimmt das statistische Sicherheitsniveau der Verlustschätzung.
4. **VaR-Haltedauer:** Anzahl der Tage, für die der mögliche Verlust geschätzt wird.
5. **Anzahl der Simulationen:** Mehr Durchläufe machen Monte-Carlo-Ergebnisse meist stabiler, brauchen aber länger.
6. **Simulationszeitraum:** Legt fest, wie viele Handelstage simuliert werden.
7. **Rendite-Faktor μ:** 0 blendet die historische Rendite aus; 1 nutzt sie unverändert; über 1 verstärkt sie sich.
8. **Volatilitäts-Faktor σ:** 0 blendet Schwankungen aus; 1 nutzt die